Ders 3 Çalışma Özeti

Ders 3 Çalışma Özeti

Genel Konular

- Asıl ders yaklaşık 00:08'de başlar. 00:59-01:16 arası ara ve sohbettir.
  - Teorik kısım İngilizce anlatıldı. Hoca önemli yerleri Türkçe tekrar etti.
- IntelliJ kullanılabilir, ama UML çizimi ve koddan diyagram çıkarma için NetBeans'teki Easy UML daha rahat.
- Nesne ve sınıf (object, class)
  - Nesne somut, sınıf soyuttur. Sınıf bir şablondur (template); nesne bu şablondan üretilen örnektir (instance).
  - Nesnenin iki parçası: özellikler (attribute, field, member) ve görevler (task, method).
- Kapsülleme (encapsulation) ve bilgi gizleme (information hiding)
- UML gösterimleri: sınıf diyagramı, nesne diyagramı, etkileşim (sequence) diyagramı, paket diyagramı.
- Nesneye mesaj gönderme (sending message), getter ve setter metotları.
- Paketler (package) ve `import`.
- Görünürlük kuralları: `private` (-), `public` (+), `protected` (#), paket (~).
- `static` alanlar ve metotlar, `final` alanlar ve metotlar.
- Yapıcı metot (constructor), varsayılan yapıcı (default constructor), sonlandırıcı (finalizer), çöp toplayıcı.
- Metot ve yapıcı aşırı yükleme (overloading). Operatör aşırı yüklemenin Java'da olmaması.
- Nesneleri == ile karşılaştırmanın sorunu ve `equals` metodu.
- Akış kontrolü (control flow), `main` metodu ve test sınıfı.
- `Car` sınıfı örneği ve `this` anahtar kelimesi.

Hocanın Özellikle Vurguladığı Kısımlar

- **Bu dersin amacı Java öğrenmek değil, nesnesel düşünmektir.** Teknoparktaki firmalar öğrencilerin tasarım kalıplarını ve nesne yönelimli tasarımı bilmediğinden şikâyet ediyor.
- **UML önemlidir.** UML çizmeyi, verilen bir UML'in eksiklerini doldurmayı ve verilen sınıf diyagramını kodlamayı bilin.
- **UML aracı olarak kod üretebilen (code generator) bir araç kullanın.** Önce UML'i çizin, sonra kod iskeletini üretin, metotların içini doldurun.
- **Alanları `private` yapın.** UML verilmemişse sınıf alanlarını `private` tanımlayın, erişimi `public` getter/setter ile sağlayın.
- **Nesne oluştururken iki operatör çalışır:** önce `new` nesneyi oluşturur ve referansını döndürür, sonra atama operatörü bu referansı değişkene yazar.
- **Nesneleri == ile karşılaştırmayın.** İçerikleri aynı olsa bile farklı nesnelerin referansları farklıdır; sonuç `false` olur. İçerik karşılaştırması için `equals` metodu kendi sınıfınıza göre tanımlanır.
- **Tasarımda ileriyi düşünün.** Yazarın adını kitabın içine gömerseniz, yazara yeni bir bilgi eklenince tasarım bozulur. Yazarı ayrı nesne yapın.
- **Değişmemesi gereken alan için setter yazmayın.** Şasi numarası gibi alanlar yalnızca yapıcıda atanmalı.
- **`main` içinde kendi sınıfından nesne üretmek özyineleme (recursion) değildir.** Bu sık yapılan bir yanlış anlamadır.

Kısa Tekrar Notları

- Nesne: somut (concrete). Sınıf: soyut (abstract), şablon. Hoca benzetmesi: sınıf kurabiye kalıbı, nesne kurabiye.
- Nesnenin durumu (state): bütün alanlarının o andaki değerleri.
- `O`, `C` sınıfının bir nesnesiyse `O`, `C`'nin bir örneğidir (instance).
- Sınıf, yeni bir veri tipi tanımlamaktır.
- Alanlar iki türlü olabilir: değer tipli (primitive: `int`, `float`, `char`) ve referans tipli (başka sınıflar: `Author`, `String`).
- Java'da `new` ile oluşturulan her şey referans tiplidir. Nesne heap'te durur, referansı stack'teki değişkende tutulur.
- Aynı değerlere sahip iki nesne yine de bellekte farklı yerlerde duran farklı nesnelerdir.
- Kapsülleme: veri ve metotlar bir arada, ayrılmaz.
- Bilgi gizleme: Nesnenin iç ayrıntıları dışarıdan gizlenir. Kullanan nesnenin karşı tarafın iç yapısını bilmesi gerekmez.
- UML sınıf diyagramı üç bölümdür: sınıf adı, alanlar, metotlar.
- UML'de alan yazımı: `- plate : String`. Önce ad, sonra iki nokta, sonra tip (C/Java'nın tersi).
- UML'de metot yazımı: `+ getPlate() : String`, `+ setPlate(plate : String)`.
- Görünürlük işaretleri: `-` private, `+` public, `#` protected, `~` paket (Java'da varsayılan).
- UML'de `static` üye altı çizili yazılır. Soyut sınıf/metot `abstract` anahtar sözcüğüyle gösterilir.
- UML nesne diyagramı: `nesneAdi : SinifAdi`, altı çizili.
- Mesaj göndermek = nesnenin metodunu çağırmak. UML'deki `kit.getPlate()` mesajının Java karşılığı `kit.getPlate();`.
- Getter: alanın değerini döndürür (`getName`). Setter: alana değer atar (`setName`). IDE'ler bunları otomatik üretir.
- Paket içindeki tüm sınıflar: `import paket1.*;`. Tek sınıf: `import paket1.ClassA;`. Alt paketler `*` ile gelmez, ayrıca import edilir.
- Farklı paketlerde aynı adlı sınıf olabilir: `java.io.File` gibi.
- `static` alan: sınıfın bütün nesnelerinin paylaştığı ortak alan. Erişim: `SinifAdi.alan`.
- `static` metot: nesne oluşturmadan `SinifAdi.metot()` ile çağrılır.
- `final` alan: bir kez atanır, sonra değişmez (C'deki `const` gibi). Genellikle yapıcıda atanır.
- `final` metot: alt sınıfta override edilemez.
- Yapıcı: sınıfla aynı adı taşır, dönüş tipi yoktur (`void` de yazılmaz), `new` ile tetiklenir.
- Yapıcı yazılmazsa parametresiz varsayılan yapıcı otomatik eklenir. Parametreli yapıcı yazılırsa varsayılan yapıcı eklenmez.
- Varsayılan değerler: `int` 0, `double`/`float` 0.0, `boolean` false, referans tipler `null`.
- Aşırı yükleme (overloading): aynı adlı metotların parametre sayısı, tipi veya tip sırası farklıdır. Yalnızca dönüş tipini değiştirmek yetmez.
- Java'da operatör aşırı yükleme yoktur (C++ ve C#'ta vardır).
- Bütün Java sınıfları dolaylı olarak `Object` sınıfından türer. `equals` metodu oradan gelir.
- `main` metodu `static`'tir; program başlarken henüz nesne olmadığı için böyle olmak zorundadır.
- `String[] args` ile programa başlangıç parametreleri verilir. Eleman sayısı `args.length` ile alınır (C'deki `argc` gerekmez).
- `this`: metodu çalıştıran nesnenin kendisi. Parametre ile alan aynı adı taşıyınca alanı belirtmek için kullanılır: `this.plate = plate;`.

Detaylı Açıklamalar

Nesneler, sınıflar ve üyeler

- Nesne, bir şeyin özelliklerini (renk, boy, uzunluk) ve görevlerini (çalış, kitapları listele) bir arada tutar.
- C'deki `struct` yalnızca özellik tutar (ör. karmaşık sayının reel ve sanal kısmı). Nesne bunlara ek olarak metot da içerir.
- Sınıf içinde tanımlı alanlara sınıfın bütün metotları doğrudan erişebilir. Yapısal programlamada bunun için değişkenin global olması gerekirdi.
- Bir nesnenin genellikle onu tekil olarak tanımlayan bir alanı olur: araba için plaka, kişi

için TC kimlik numarası, öğrenci için öğrenci numarası.
- Kitap örneği: `int id`, `float price`, `int numberOfPages`, `Author author`. Yazarı `String` tutmak nesne yönelimli tasarıma uygun değildir.

Nesne oluşturma ve bellek

- `Book b;` yalnızca bir referans değişkeni tanımlar. Henüz nesne yoktur.
- `b = new Book();` önce `new` ile heap'te nesne oluşturulur ve yapıcı çalışır. Sonra nesnenin referansı `b`'ye atanır.
- Aynı sıra metot çağrısında da geçerlidir: `f(new Book());` önce nesne oluşturulur, sonra referansı metoda geçirilir.
- JVM her nesne için ayırt edici bir referans tutar. Bu işlemler programcıya görünmez; C'deki gibi işaretçiye müdahale edemezsiniz.
- C'de `malloc` ile ayrılan yer `free` ile bırakılır. Java'da bunu çöp toplayıcı (garbage collector) yapar.
- Örnek: `Book b = new Book(); ... b = new Book();` sonrası ilk nesneyi gösteren referans kalmaz. Çöp toplayıcı bir süre sonra onu temizler.

Bilgi gizleme ve görünürlük

- Günlük hayatta nesnelerin bütün özelliklerini doğrudan kullanmayız.
  - Monitörü açmak için düğmeye basarız; renk karışımı gibi iç işlevlere erişmeyiz.
  - Lambayı yakmak için kabloları birbirine değdirmeyiz, anahtar kullanırız.
  - Televizyon izlemek için kumandadaki güç, ses ve kanal tuşlarını bilmek yeter.
  - Arkadaşınıza borç verirken maaşınızı veya PIN numaranızı söylemezsiniz.
- Bu iç ayrıntılara erişim gereksizdir ve yasaklanmalıdır. Erişim nesnenin işleyişini bozabilir.
- Uygulamada bilgi gizleme kusursuz uygulanamaz: Bir sınıftaki değişiklik başka sınıfları da etkileyebilir. İlkeye ne kadar uyulursa değişiklikten etkilenen sınıf sayısı o kadar azalır.
- Kural: Alanlar `private`, onlara erişen yardımcı metotlar (getter/setter) `public`.
- Bir metoda erişimi kısıtlamak için metodu silmek yerine görünürlüğünü `private` yapmak tercih edilir.

Nesneye mesaj gönderme

- Amaç nesneye bir iş yaptırmak (kitapları yazdır, fiyat hesapla) veya bir üyesine erişmek (adı al, adı değiştir).
- Nesne yönelimli program, nesneler arası mesaj alışverişiyle çalışır.
- UML etkileşim diyagramı örneği: `client` nesnesi `kit : Car` nesnesine `getPlate()` mesajı gönderir. `kit` cevap olarak plakasını döndürür.
  - Cevabın tipi tasarıma bağlıdır: `String`, `int`, dizi veya özel bir sınıf olabilir.
- Metoda gönderilen parametre, tanımdaki tiple uyumlu olmalıdır. İleride görülecek kalıtım ilişkisi sayesinde aynı tip olması gerekmeyebilir.

Paketler

- Projeyi mantıksal parçalara bölmek ve tip çakışmalarını önlemek için kullanılır.
- Örnek: Bir hastane yazılımında muhasebe ve doktor işlemleri ayrı paketlerde tutulur. Her ikisinde "bilgi listeleme" sınıfı olabilir.
- Paketler soyut kaplardır (abstract containers). Gruplamanın yanında yetkilendirmeyi de sağlar.

`static` örneği: kantin

- Kantin günde 1000 kişiye hizmet verecek. 1000. müşteri gelince bir bayrak değişir; bütün görevliler bu ortak bayrağa bakıp işi bitirir.
- Bu bayrak `static` tanımlanır. Görevli nesneleri farklıdır, ama hepsi aynı alanı görür.
- Bayrak, müşteri ekleme işini yapan kantin sınıfında tutulur. Öğrenciler bu alanı değiştiremez.

- C'deki `static` yerel değişken de fonksiyon tekrar çağrılınca değerini korur. Java'daki `static` alan ise nesneye değil sınıfa aittir.
- Başka bir örnek: Her otomobilin 4 tekerleği vardır; bu ortak bir özelliktir.

Yapıcı ve aşırı yükleme

- Yapıcı, nesne oluşturulurken alanlara başlangıç değeri vermek için kullanılır. Örnek: `new Book(1, "Java")`.
- `final` alanlara değer atamak için en uygun yer yapıcıdır.
- Aynı sınıfta imzası farklı birden fazla yapıcı olabilir: `Car(int i)`, `Car(String s)`, `Car(int i, String s)`.
- Metot aşırı yükleme örneği: aynı adlı `calculateArea` metodu tek parametreyle kare, iki parametreyle dikdörtgen alanı hesaplar.
- `PrintStream` sınıfındaki `println` metodunun `char[]`, `int`, `long`, `String`, `Object` gibi parametreli birçok sürümü vardır. IDE'de `println` yazınca bu yüzden çok seçenek çıkar.

`==` ve `equals`

- `Book b = new Book(1, "Java"); Book b1 = new Book(1, "Java");` sonrası `b == b1` her zaman `false` olur. Çünkü iki değişken farklı adreslerdeki nesneleri gösterir.
- İçerikleri karşılaştırmak için alanlar tek tek karşılaştırılmalıdır. Java bunun için `Object` sınıfından gelen `equals` metodunu sunar. Bu metot sınıfın ihtiyacına göre yeniden tanımlanır (override).
- Hoca `equals` metodunun nasıl yazılacağını ileride kalıtım konusunda anlatacağını söyledi. Doğru bir taslak (Java ile denendi):

```java
@Override
public boolean equals(Object o) {
    if (!(o instanceof Kitap)) return false;
    Kitap k = (Kitap) o;
    return this.id == k.id && this.ad.equals(k.ad);
}
```

Akış kontrolü ve `main`

- Yapısal programlamada akış ilk satırdan son satıra gider; araya fonksiyon çağrıları girer.
- Nesne yönelimli programda akış sırasında nesne oluşturma ve metot çağrıları olur. Kodun yazılış sırası ile çalışma sırası genellikle farklıdır.
- Sınıf içinde alanların, yapıcının ve metotların sırası zorunlu değildir. Okunabilirlik için önce alanlar, sonra yapıcı, sonra metotlar yazılır.
- Her projede `main` metodu olan bir test (driver) sınıfı olur. `main`'in görevi ilk nesneleri oluşturmak ve programı başlatmaktır.

`Car` sınıfı örneği

- Slayttaki ilk sürüm (kendi `main` metodunu içerir):

```java
package nyp01a;
public class Car {
    private String plate;
    public Car(String plateNr) {
        plate = plateNr;
    }
    public String getPlate() {
        return plate;
    }
    public void setPlate(String plate) {
        this.plate = plate;
    }
    public void introduceSelf() {
        System.out.println("My plate: " + getPlate());
```

```
    }
    public static void main(String[] args) {
        Car aCar;
        aCar = new Car("34 RA 440");
        aCar.introduceSelf();
    }
}
```

- `introduceSelf` içinde `plate` doğrudan da yazılabilirdi. Modülerlik için `getPlate()` kullanıldı.
- `main` aynı sınıfın içinde olduğu için `aCar.plate` ile doğrudan erişim de mümkündür. Başka bir sınıftan bu alana erişilemez; yapıcı veya getter/setter gerekir.
- İkinci sürüm: `private String chassisNR` alanı eklendi. Yapıcı `Car(String plateNr, String chassisNR)` iki değeri birlikte alır. Şasi numarası için yalnızca `getChassisNR()` var, setter yok. Bu sürümde `main` yoktur; test için ayrı bir sınıf yazılmalıdır.
- Doğru tasarım: Gerçek hayatta her araba üretilirken plaka ve şasi numarası alır. İkisi aynı yapıcıda atanmalıdır.
- Slayttaki yanlış tasarım: Aynı imzalı iki yapıcı (`Car(String)` ve `Car(String)`) yazılmıştı. Bu derlenmez. Derlenseydi de tasarım yanlış olurdu.
- `this`: Yapıcı veya metodun içinde, o anda üzerinde çalışılan nesneyi gösterir. Dışarıda `c.getPlate()` çağrıldığında, metodun içindeki `this` `c` nesnesidir. Parametre ile alan aynı adı taşıyınca karışıklığı önler.

Ders sonrası notlar

- **`static` erişim kuralı ters söylendi:** Hoca "static üyeleri static olmayan bir metotta kullanırsanız derleme hatası alırsınız" dedi. Doğrusu tersidir. Static olmayan metotlar static üyelere erişebilir. Static bir metot ise sınıfın static olmayan (nesneye ait) alan ve metotlarına doğrudan erişemez; derleme hatası verir.
- **UML'de `void` dönüş tipi:** Hoca, metot `void` döndürüyorsa dönüş tipi eklemenin "yasak" olduğunu söyledi. UML'de `void` dönüş tipi genellikle yazılmaz, ama `: void` yazmak yanlış değildir.
- **`char` varsayılan değeri:** Hoca `char` için önce "null", hemen sonra "sıfır gibi" dedi. `char` değer tiplidir; varsayılan değeri `'\u0000'` karakteridir (sayısal değeri 0). `null` yalnızca referans tipler içindir. Java ile denendi: `(int) b1.c` sonucu `0`.
- **`null` ve `new`:** Bir öğrencinin sorusuna hoca "new ile üretilen her şey null tipindedir" dedi. Doğrusu: Değer atanmamış referans tipli **alanlar** varsayılan olarak `null` olur. `new` ile üretilen nesne `null` değildir.
- **`final` metot:** Hoca Türkçe özetinde `final` metotların "kalıtımla türetilemez" olduğunu da söyledi. `final` metot alt sınıfa geçer ve alt sınıf nesnesiyle çağrılır. Engellenen yalnız override işlemidir. Java ile denendi.
- **Görünürlük değişikliği:** Hoca üyeyi `public`'ten `private`'a çevirmenin hata vereceğini, tersinin de ("vice versa") vereceğini söyledi. `private`'tan `public`'e geçiş hata vermez.
- **`this` "global" değildir:** Hoca `this` için "globaldeki değişkene erişmeyi sağlar" dedi. `this.plate`, nesnenin alanını (instance field) gösterir. Java'da global değişken yoktur.
- **Yapıcılar her zaman `public` olmak zorunda değil:** Slaytta "yapıcılar public'tir" deniyor. Yapıcı `private` veya `protected` da olabilir. Hoca da `PrintStream` sınıfında `private` bir yapıcı gösterdi. `private` yapıcı, dışarıdan nesne oluşturmayı engellemek için kullanılır.
- **Finalizer:** Hoca `finalize` metodunu kendimiz çağırarak çöp toplayıcıyı tetikleyebileceğimizi söyledi. `finalize` çöp toplayıcıyı tetiklemez. Çöp toplama için `System.gc()` ile istekte bulunulur; JVM'nin bunu hemen yapacağının garantisi yoktur. `finalize` Java 9'dan beri kullanımdan kaldırılmış (deprecated) durumdadır.
- **`equals` dönüş tipi:** Hoca derste `equals` için "return 1 / return 0" yazdı. Java'da `Object.equals` `boolean` döndürür (`true`/`false`). `public int equals(Book b)` derlenir ama override değil, ayrı bir aşırı yüklemedir; `Object` tipli referansla yapılan karşılaştırmada çağrılmaz. Ayrıca `String` alanlar `==` ile değil `equals` ile karşılaştırılmalıdır.
- **`==` ve sarmalayıcı sınıflar:** Hoca sohbetteki bir yorumu okuyarak `Integer` ve `String` için `==`'ın çalıştığını aktardı, ikisine de wrapper sınıf dedi. Bu yanıltıcıdır. `String` sarmalayıcı sınıf değildir; sarmalayıcılar ilkel tiplere karşılık gelir. Aynı metin sabitleri (`"Java" == "Java"`) ve -128..127 arasındaki `Integer` değerleri önbellekten aynı nesneyi gösterdiği

için `true` çıkabilir. `new` ile oluşturulan nesnelerde sonuç `false` olur. İçerik karşılaştırması için her zaman `equals` kullanın.
- **Yanlış yapıcı örneği:** Hoca yanlış örneği önce “execution error”, sonra “compile time error” diye nitelendi. Aynı imzalı iki yapıcı derleme hatasıdır.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.